Tractor Trace Format Description.

Files:

1. App (tractor_t.app[.gz])
2. Common (tractor_t.com.<rank>[.gz])
3. PTPD output (ptpd-tractor_t.log)
4. Maps (maps.tractor_t.com.<rank>)
5. Main (tractor_t.mpi.<rank>[.gz])
6. Hardware (counters) file (tractor_t.hwc.<rank>[.gz])

Format descriptions

HWC Format description:

1. Each record is a single line. Field separator is ':'.
2. Each record corresponds a single mpi call with one exception: mpi_sendrecv and mpi_sendrecv_replace. These calls occupy 2 records.
3. The format is as follows:

   ids_section:TSCs_section:address_section:hardware_counters_section:memprofile_section

   a. ids_section. id_columns ':' separated decimal signed values (id_columns is taken from the PEAT_FRT_NumberOfEvents record of the common file, it is the same for all ranks of a trace and never less than 2). id_0 is the id of the top-level record: started from 1, incremental per rank. id_1…id_(n-1) form the path of a nested record (an MPI call the MPI library issued from inside another traced call): the index of the record among the children of its parent at each depth, -1 where unused. A top-level record has -1 in all columns but the first. The same ids open the corresponding line of the main (mpi) file; both files are line aligned.

   b. thread_id is not present in this file (it is on the main file line only).

   c. TSC_Section: a ':' separated values. Number of values are taken from num_tscs read from app file (4). Their meaning is the same as in the fabric records: [0] start of the record relative to the start of the previous record in the file (the previous line, whatever its nesting level; the very first record is relative to 0); [1] tool prolog: MPI entry minus record start; [2] tool epilog: record exit minus MPI exit; [3] duration: record exit minus record start. Pseudocode is as follows (tsc_val_base=0 at the very beginning):

```
tsc_start = read_val(id_columns + 0) + tsc_val_base;

tsc_val_base = tsc_start; // saved for the next record
```

mpi_in = tsc_start + read_val(id_columns + 1);

func_out = tsc_start + read_val(id_columns + 3);

mpi_out = func_out - read_val(id_columns + 2);

MPI_Sendrecv and MPI_Sendrecv_replace (and their MPIC_ counterparts) occupy two lines with the same ids: the second line carries the receive part in the main file, its TSC values are all 0 (same start, no own timing) and it does not advance tsc_val_base.

d. Address_section: a':' separated values. Number of values are taken from num_addrs read from app file (see below). The values here are the IDs of real addresses which may be read from app file. Pseudocode is as follows:

```
for (int addr_ind=0;addr_ind<num_addrs;addr_ind++) {
 addr_id=read_val(id_columns + num_tscs + addr_ind);
}
```

e. Hardware_counters_section is not fully described here (since its usage is very narrow). It's a ':' separated section particularly with 4 values (2 pairs): instruction_retired and clockticks before mpi call (i.e. in user section preceding mpi call) and during mpi call.

f. Memprofile_section is NOT described here.

Main MPI Format description:

4. All MPI functions are divided into groups called classes. Now it's 9 classes.
    a. P2P (point to point (or peer to peer))
    b. PROBE (mpi_probe etc)
    c. REQUEST (MPI_WAIT/TEST etc)
    d. COLLECTIVE (MPI_BCAST etc)
    e. COMM or COMM_COLLECTIVE (communicator) MPI_COMM_CREATE etc.
    f. OSC (One Sided Communication) MPI_GET/PUT/ACCUMULATE etc
    g. FIO (File IO)
    h. BUFFERED_LOCAL (MPI_PACK/UNPACK)

i. LOCAL (all other)

5. record divided into 4 parts: fixed, class dependent, flexible (function/parameter dependent), extra (special tractor build dependent)

6. Field separator is ':'. Extra part can be delimited by ';' as well.

7. Fixed part

   ids_section:thread_id:num_children:mpi_func_id:

   a. ids_section. The same ids as on the corresponding hwc line (see above).

   b. thread_id. Decimal, unsigned. OpenMP thread number, 0 if no OpenMP is involved. Every thread has its own tree of records; the ids are unique per rank, not per thread.

      num_children – decimal, signed. Number of records directly nested in this one: MPI calls the MPI library issued while this call was running (MPIC_* transfers of a collective, MPID_Request_Release, progress engine calls for another outstanding nonblocking operation and so on). The records of one thread form a pre-order tree: the num_children records following this record (each followed by its own subtree) are its children, in call order. A child's id at depth d is its position among its siblings, so the tree can also be rebuilt from the ids alone. The tractor keeps one storage per (thread, nesting level); the depth is limited by PEAT_TRACTOR_MAX_NESTING_LEVELS (default 4) and the deepest level reached defines id_columns.

   c. mpi_func_id – decimal, unsigned. Unique ID of MPI function[1].

8. Class dependent and flexible

   a. P2P and PROBE

      Class dependent part.

      :comm:peer:request:message:tag:transfer_bytes

      1. comm – decimal, signed. Global MPI Communicator ID.

      2. peer – decimal, signed. Remote rank (number is in MPI_COMM_WORLD). Can contain wildcards (MPI_ANY_SOURCE) and MPI_PROC_NULL.

      3. request – hexadecimal, unsigned. MPI Request. Valid only in case of non-blocking send/receive or persistent request. For MPI_IPROBE there is flag here. For other P2P and PROBE class function its value should be 0.

1If T_PRINT_FUNC_NAMES is turned on this value is the string representing actual MPI call (like MPI_INIT).

4. message– hexadecimal, unsigned. MPI Message. Valid only in case of send/receive/probe with message (MPI_MXXXX,MPI_ImXXXX, available in MPI3).
5. tag – decimal, signed. MPI TAG. Can contain wildcards (MPI_ANY_TAG).
6. transfer_bytes – decimal, signed. Number of transferred bytes[2].

There is no flexible part in case of P2P and PROBE class.

b. REQUEST

Class dependent part.

:input_request_num:triggered_request_num:flag:request_addr

1. input_request_num – decimal, signed. Number of input requests.
2. triggered_request_num – decimal, signed. number of request which complete during this call.
3. flag – decimal, signed. flag for MPI_TEST where applicable and -1 for any other case.
4. request_addr - hexadecimal, unsigned. Address of MPI_Requests(S).

Flexible part:

:v_0:v_1:…:v_n – hexadecimal, unsigned. Values of each request supplied to the function. N=input_request_num.

:t_{00}:t_{01}:t_{02}:t_{03}...:t_{m0}:t_{m1}:t_{m2}:t_{m3} – 4-tuple representing[3] (index(decimal, signed),tag(decimal, signed),source(decimal, signed),transferred_bytes(decimal, signed)) of each completed request. m=triggered_request_num. index=[0…input_request_num-1]

c. COLLECTIVE

Class dependent part.

:comm:root:request:send_bytes_num:recv_bytes_num

1. comm – decimal, signed. Global MPI Communicator ID.

2Meaningful only in case of actual sending or receiving. In probes its value is undefined. In case of persistent request (MPI_XXXX_Init) it shows actual value of sending or receiving bytes of corresponding MPI_STARTXXX call.

3Valid only for “receive” requests (f.i. initiated by mpi_irecv or mpi_recv_init). For send or null requests their value are undefined and should be ignored.

2. root – decimal, signed. Root process(in MPI_COMM_WORLD). -1 if no root process in this particular call.
3. send_bytes_num, recv_bytes_num - decimal, signed. Number of send and receive bytes fields.
4. Request - hexadecimal, unsigned. MPI_Request if applicable, 0 otherwise.

Flexible part:

:SB_0:SB_1:…:SB_p – decimal, signed. Bytes sent by this rank. p=send_bytes_num. Followed by :RB_0:RB_1:…:RB_q – decimal, signed. Bytes received by this rank. q=recv_bytes_num.

d. COMM

Class dependent part.

:comm:newcomm:remote_comm:inter_comm:request

comm, newcomm, remote_comm, inter_comm – decimal, signed, 0 if not applicable (not all four exist in all communicator functions). request – hexadecimal, unsigned. MPI_Request of a nonblocking communicator creation, 0 otherwise. No flexible part.

e. OSC

Class dependent part is pretty similar to P2P. All values have the same meanings except explicitly stated otherwise.

:comm:peer:request:window:size:transfer_bytes

1. size – decimal, signed. Size for window creation functions. Value should be ignored for all others.
2. window – hexadecimal, unsigned. MPI window handle.

This class has no flexible part.

f. FIO

Class dependent part.

For all functions except MPI_FILE_OPEN it's:

:offset:fh:request:transferred_bytes where

1. offset – decimal, signed. Offset if file if applicable. 0 otherwise.
2. fh – hexadecimal, unsigned. File handle.

3. request – hexadecimal, unsigned. MPIO_Request if applicable. 0 otherwise.
4. transferred_bytes – decimal, signed. Number of transferred bytes if applicable. 0 otherwise.

For MPI_FILE_OPEN class dependent part is:

:comm:fh

Meanings are the same as above.

g. Buffered Local.

:count:size

1. count – decimal, signed.
2. size – decimal, signed.

h. Local has no class dependent and flexible parts.

Legacy traces. Traces written before nesting support have two id columns primary_id:secondary_id followed by thread_id on both the main and the hwc lines, no num_children field, an additional num_colts field (number of nested records, all of depth one, listed after the call) in the REQUEST, COLLECTIVE and COMM class dependent parts, and TSC value [2] is MPI exit relative to the record start. Readers tell the two formats apart by the number of fields of the PEAT_FRT_NumberOfEvents record (see Common).

Extra part:

Printed at the end of flexible part. Now only 3 extra parts are defined:

1. Buffers addresses (only where applicable).
   a. Printed after ';'.
   b. Field separator is ':'.
   c. Can contain 4-tuples (virtual_address,size,physical_entry,flags) if VIRT2PHYS is enabled or 2-tuples (virtual_address,size) if MPIBUFFERS is enabled.
   d. If particular tuple is invalid all its values are zeros.

MAP Format description:

Copy of /proc/pid/maps

Rest files format description

These files contain just records in flexible format (i.e. format depends on record number).

RECORD_ID:RECORD.

App Format description:

PEAT File Record Type Definitions.
An informative table to make correlation between id and theirs “high level” meaning. Can be used to translate messages between different version of tractor and pet/tv. (1 = PEAT_FRT_FRT_Definitions)
1:id:description (PEAT_FRT_)

PEAT_FRT_FRAME_MARKER:decimal_value – currently not used.
PEAT_FRT_VarBlockInfo:num_tscs:num_addrs:num_hwcs – number of values used to decode hwc file.
PEAT_FRT_MPMMasterRanks:number:ranks – Master (first rank on each node) ranks. Number is decimal nonnegative value. ranks are separated by “:”.
PEAT_FRT_MPMNodesFreq:node_num:rank:freq0:freq1:init_delay:hostname. All values are decimal except hostname (string). Init_delay is now not used.
PEAT_FRT_MPMRanksAndPPN:hostname:frequency:ppn:id:ranks - mostly informational. Ranks separated by ‘,’.
PEAT_FRT_MPMAllRanksTimeStamp:nodes_num:4_tuples – Initial time (ptp) correction for all nodes. number of 4 tuples = nodes_num. (Deprecated).

PEAT_FRT_MPIFunctionNameId:id:MPI_FUNC – translation between function id (decimal) and function name (string)
PEAT_FRT_GlobalCommunicatorsReport:id:size:ranks – this id is used in comm and p2p records.
PEAT_FRT_Sync_Point:size:<values> - Synchronization point. To be used if works without PTPD. Usually occurs twice (at the beginning and end of trace time).

Common Format description:

PEAT_FRT_AppStartStopInfo:<start_tscval>:<stop_tscval>:<duration>
PEAT_FRT_CpuFreq:0:<freq>:<freq>
PEAT_FRT_NumberOfEvents:number_of_events:max_top_level_id:max_thread_id:id_columns:max_fabric_id – all decimal. number_of_events counts the written records (nested ones included); id_columns is the number of id columns of the main and hwc files, max(2, deepest nesting level + 1), agreed among all ranks at MPI_Finalize; max_fabric_id is the last id of the fabric file. Legacy traces have 4 values here (number_of_events:max_id:max_thread_id:max_fabric_id).
PEAT_FRT_CollectorFrameInfo: - currently reserved (not used)
PEAT_FRT_UserAddresses:id:bin_address:binary_file:function:source_file:line
id, line – decimal
bin_addres – hexadecimal
All the rest – strings

PEAT_FRT_MPIPredefined:MPI_SUCCESS:MPI_UNDEFINED:MPI_COMM_WORLD: MPI_COMM_SELF: MPI_COMM_NULL: MPI_PROC_NULL:MPI_ANY_TAG:MPI_ANY_SOURCE:MPI_REQUEST_NULL

PTPD Format description:

PEAT_FRT_PTPMode:<dedicated_mode_in_use> (1 or 0)
PEAT_FRT_PTPRanks: <number_of_ranks>:<rank_to_comm_world_mappings> - 'masters' rank mapping record

PEAT_FRT_PTPNodesFreq: :<rank_id>:<freq1>:<freq2>:<hostname> - frequency and hostname record
PEAT_FRT_PTPMainRecord:<destination_rank>:t1:t2:t3:t4 - basic PTP record.

Bind files (written by mat and pet)

Bind files (tractor_t.bind.<rank>.out[.gz]) are not written by the tractor, they are produced by mat and pet from a trace and read by pet and qTV. They refer to trace records by rank and full id path, so their width follows id_columns of the trace. Field separator is ':', RECORD_ID first.

PEAT_RT_Bind_Peer:function:rank:ids:function:rank:ids:size – a send record bound to its receive record (ids = id_columns values each).

PEAT_RT_Bind_Completion:function:ids:function:ids and PEAT_RT_Bind_Initiation:function:ids:function:ids – a nonblocking or persistent record and the completing (starting) call of the same rank.

PEAT_RT_Bind_Collective:function:rank:ids:rank:ids:… – the member records of one collective call, one rank:ids pair per participating rank.